Task Manager Application - Assignment

Task Overview

In this assignment, you will build a fully interactive **Task Manager Application** using only **HTML, CSS, and Vanilla JavaScript**.

The purpose of this project is to strengthen your understanding of:

- HTML → Browser Rendering Pipeline
- Parsing
- Tokenization
- DOM Tree
- CSSOM Tree
- Render Tree
- Attributes vs Properties
- DOM Manipulation
- Event Handling
- Event Propagation (Bubbling & Capturing)
- Event Delegation

⚠ *Frameworks and libraries are not allowed. Everything must be built using DOM APIs and Vanilla JavaScript.*

Learning Objectives

By completing this project, you should be able to:

- Dynamically create and remove DOM elements
- Understand the difference between Attributes and Properties
- Use Event Delegation effectively
- Demonstrate Event Bubbling and Event Capturing
- Build real-world UI interactions using DOM APIs
- Explain how a browser converts HTML and CSS into a Render Tree

---

Features to Implement

1. Task Creation Module

Create a form containing:

- Task Title Input
- Category Dropdown
- Add Task Button

When the user submits the form:

- Create a new task card dynamically

Use:

- createElement()
- createTextNode()
- append() or appendChild()

Each task should appear instantly without page refresh.

2. Attributes vs Properties

Every task card must contain custom data attributes:

```
data-id
data-status
data-category
```

Practice using:

- getAttribute()
- setAttribute()
- removeAttribute()
- hasAttribute()
- dataset

Demonstration Required

Show the difference between:

```
input.value
```

and

```
input.getAttribute("value")
```

Add comments explaining the difference.

3. DOM Manipulation

Each task card should contain:

- Edit Button
- Complete Button
- Delete Button

Use the following methods somewhere in the project:

```
append()
prepend()
before()
after()
replaceWith()
remove()
```

4. Theme Toggle

Create a Dark Mode / Light Mode switch.

Requirements:

- Use classList
- Use dataset

- Use setAttribute()

Store the current theme inside a custom data attribute. Example:

```
data-theme="dark"
```

5. Event Handling

Implement functionality for:

- Add Task
- Delete Task
- Edit Task
- Complete Task

Use:

```
addEventListener()
```

6. Event Delegation

Instead of attaching separate event listeners to every task card:

- Attach a single listener to the parent container
- Handle actions using Event Delegation

7. Event Propagation Demonstration

Create the following structure:

```
Grandparent
 └── Parent
      └── Child Button
```

Demonstrate:

Event Bubbling

Log execution order in console. Example:

```
Child
Parent
Grandparent
```

Event Capturing

Log execution order in console. Example:

```
Grandparent
Parent
Child
```

Explain the difference in comments.

8. Browser Rendering Pipeline Section

Create a separate visual section on the webpage explaining:

```
HTML
 ↓
Parsing
```

```
 ↓
Tokenization
 ↓
DOM Tree

CSS
 ↓
CSSOM Tree

DOM Tree + CSSOM Tree
 ↓
Render Tree
```

Display this flow using:

- Cards
- Flow Diagram
- Timeline
- Boxes connected with arrows

Bonus Features

Complete any of the following:

- Task Search
- Task Filter by Category
- Completed Task Counter
- Pending Task Counter
- Clear All Tasks Button
- Use DocumentFragment
- Local Storage Integration

Submission Requirements

Submit the following:

1. GitHub Repository

Push complete source code.

2. Live Deployment

Deploy the project using:

- Netlify
- Vercel
- GitHub Pages

3. README File

README must explain:

- Parsing
- Tokenization
- DOM Tree
- CSSOM Tree
- Render Tree
- Event Bubbling
- Event Capturing
- Event Delegation

Evaluation Criteria

Criteria	Weightage
DOM Manipulation	30%
Event Handling & Delegation	25%
Attributes vs Properties	15%
Code Quality	15%
UI/UX	15%

Expected Outcome

By the end of this assignment, you should have a complete Task Manager Application that demonstrates real-world usage of DOM APIs, Event Propagation concepts, Attributes vs Properties, and the Browser Rendering Pipeline.